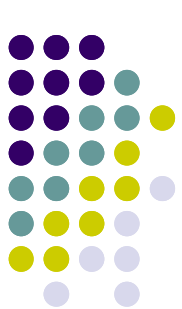
UML Diagramming

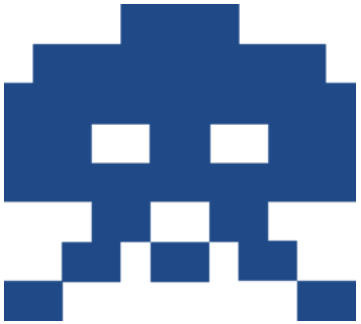


Architecture of Computer Games

SE 456

Ed Keenan

10 January 2024



13.0.11.3.17 Mayan Long Count

Overview

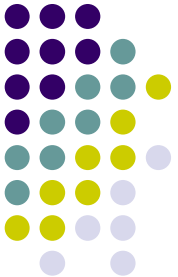
- Types of documentation
  - Coding
  - Design
- UML
- Composition
- Aggregation

Documentation

- Two different types
  - ***Source code*** documentation
    - How did you implement that method?
    - What assumptions or tricks are you using?
  - ***Whole Design*** documentation
    - What design patterns
    - How do blocks of code work together
    - Philosophy
    - Style

Design Documentation:

UML



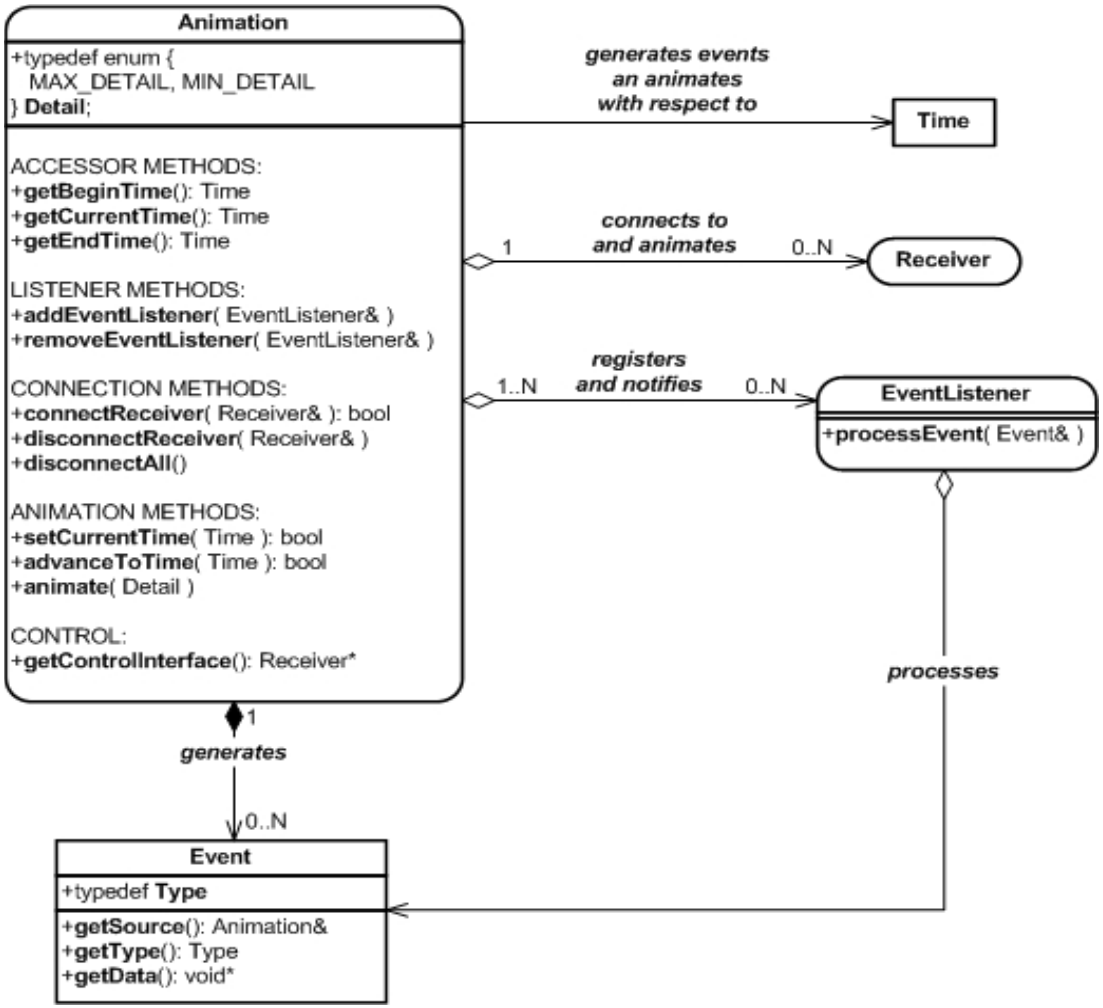
- Unified Modeling Language
  - Family of graphical notations that help describe and ***define*** software systems.
- Types:
  - Class Diagrams
  - Sequence Diagrams
  - Use Case Diagrams
  - State Machine Diagrams
  - Activity Diagrams
  - And more…

Design Documentation:

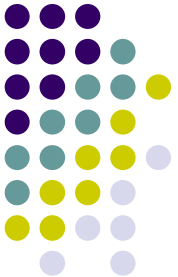
UML Class Diagrams

- Class diagram
  - describes the types of objects in the system
  - kinds of static relationships that exist among them
- Communicate Class's
  - properties
  - operations
  - constraints
  - "the way objects are connected"

Design Documentation:
Class Diagrams



Composition vs Aggregation



- A lot of confusion, but the meaning is important to convey
- Key point of view
  - Who owns the object?

Composition

- Inheritance gives us an 'is-a' relationship. (maybe)
- *Composition* gives us a ***part-of*** relationship. Composition is shown on a UML diagram as a filled diamond
- Model of a car
  - An engine is part-of a car.
  - Lifetime of the part (Engine) is managed by the whole (Car)
- When Car is destroyed, Engine is destroyed along with it.

```
public class Engine
      {
       . . .
      }
public class Car
{
    Engine e = new Engine();
    .......
}
```

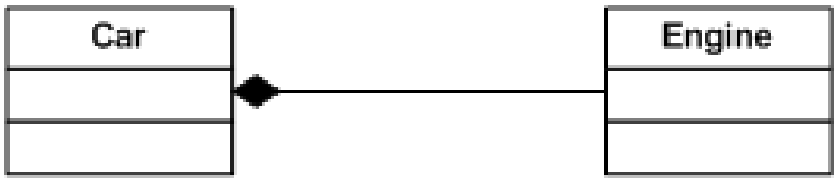


Figure 1 - Composition

- Car manages the lifetime of Engine.

Aggregation

- ***Aggregation*** gives us a ***has-a*** relationship.
- A database of customers and a separate database that holds all addresses
  - Aggregation would make sense in this situation, as a Customer 'has-a' Address.
  - if the customer ceases to exist, does the address?

```
public class Address
{
      . . .
}
public class Person
{
    private Address address;
    public Person(Address address)
    {
        this.address = address;
    }
}
```

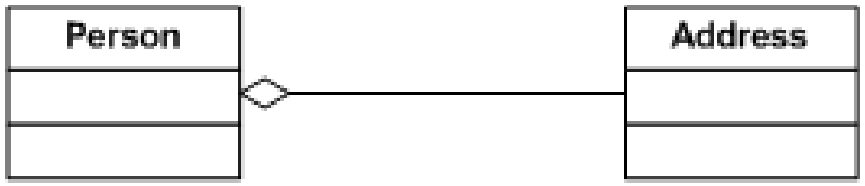


Figure 2 - Aggregation

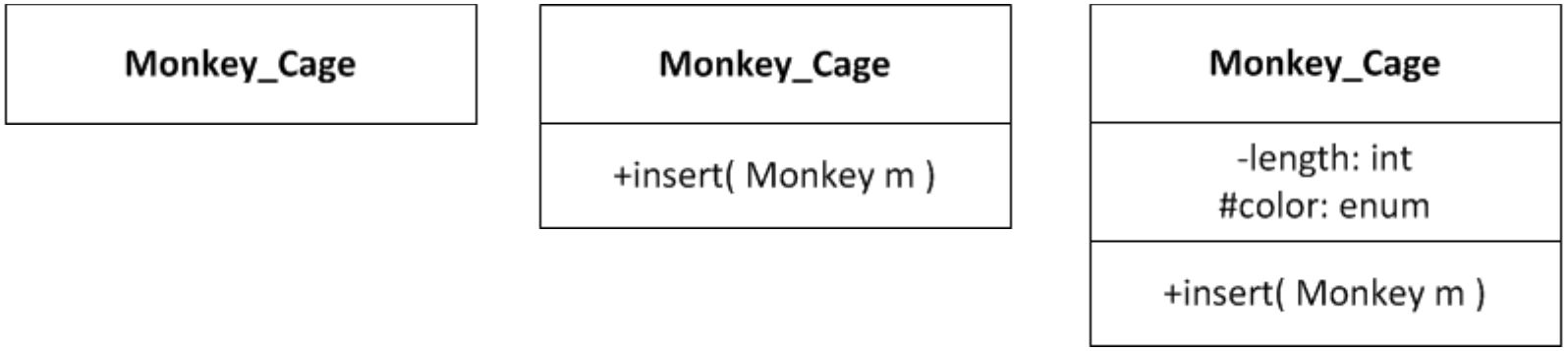
- Person would then be used as follows:

```
Address address = new Address();
Person person = new Person(address);
```

- Person does not manage the lifetime of Address. If Person is destroyed, the Address still exists.

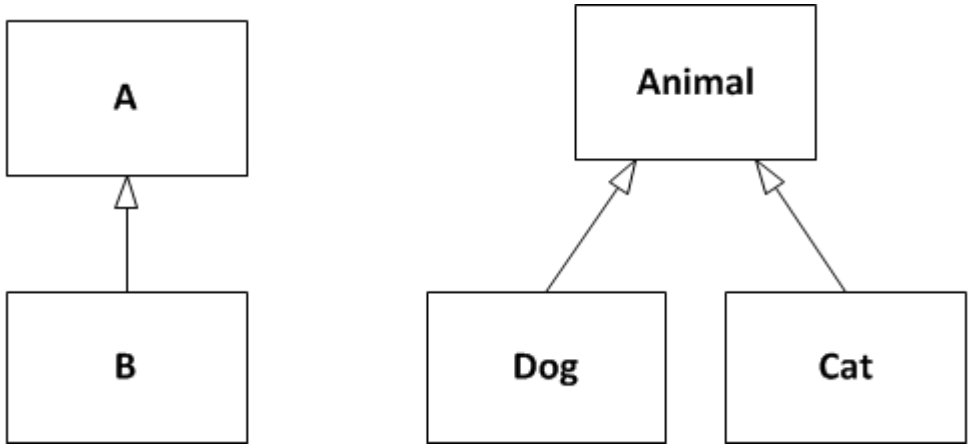
UML Notation for Access

- Public Access uses a plus sign (+)
- Protected Access uses a pound sign (#)
- Private Access uses a minus sign (-)

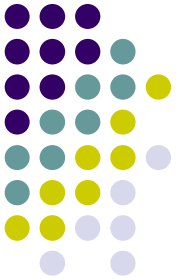


UML Notation for Inheritance

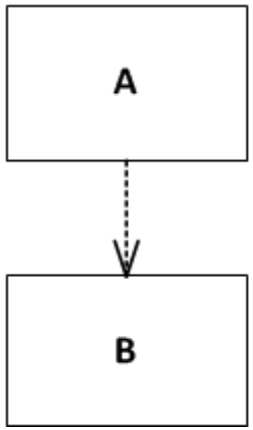
- Inheritance
  - Points to the parent
  - B is derived from A

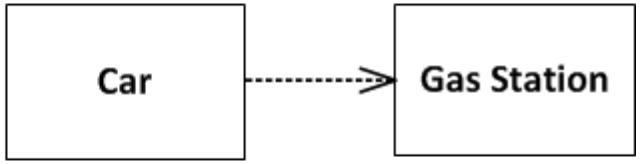


UML Notation for Dependency



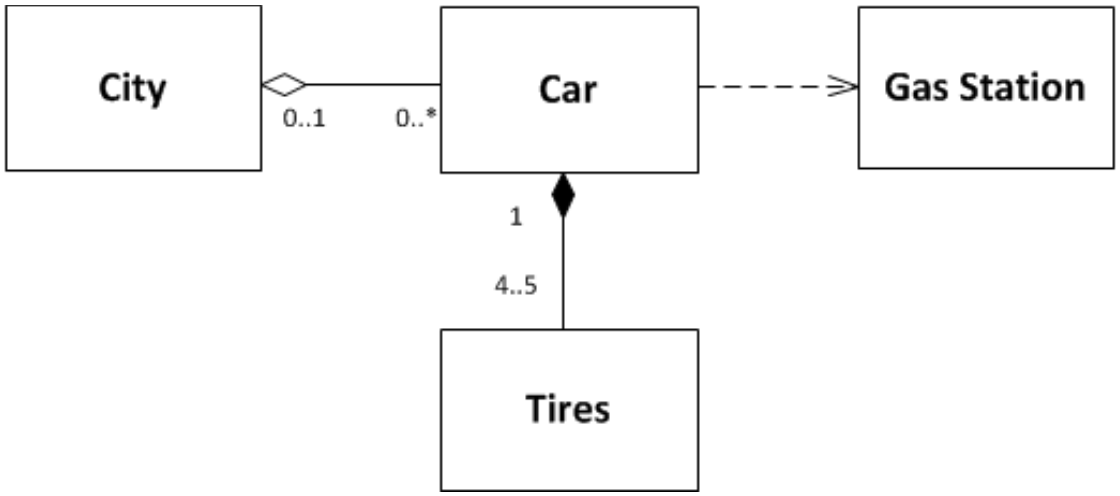
- Dependency
  - Points to the it's buddy
  - A depends on B
  - A uses B





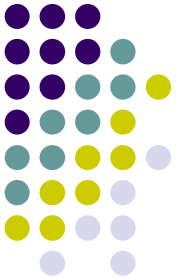
UML Notation for Cardinality

- Cardinality
  - Number associated
  - * - any number



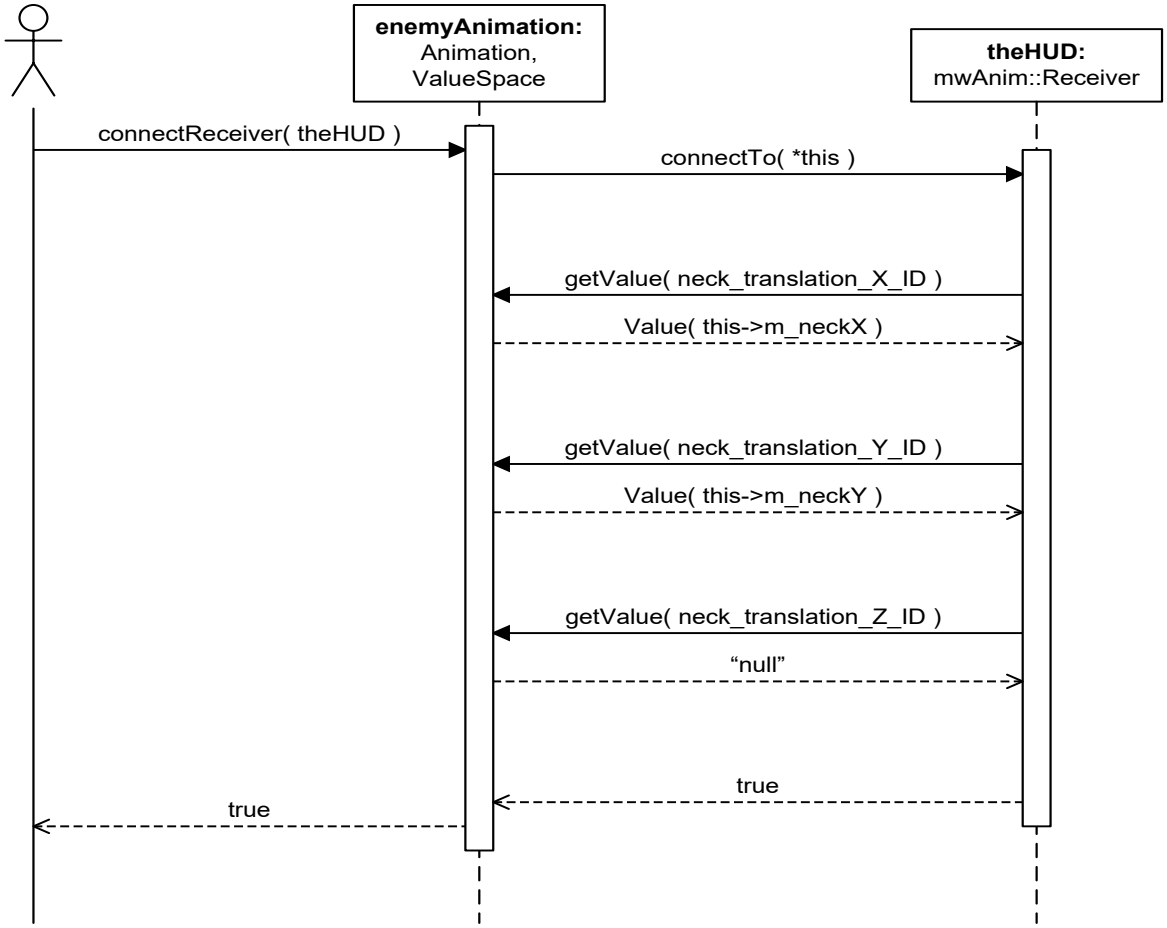
Design Documentation:

UML Sequence Diagrams



- Interaction diagrams
  - Describe how groups of objects collaborate in some behavior
  - The UML defines several forms of interaction diagram
    - Most common is the sequence diagram.
- Sequence diagram
  - Captures the behavior of a single scenario
  - Shows a number of example objects and the messages that are passed between these objects within the use case.
  - Single use case

Design Documentation:
Sequence Diagrams

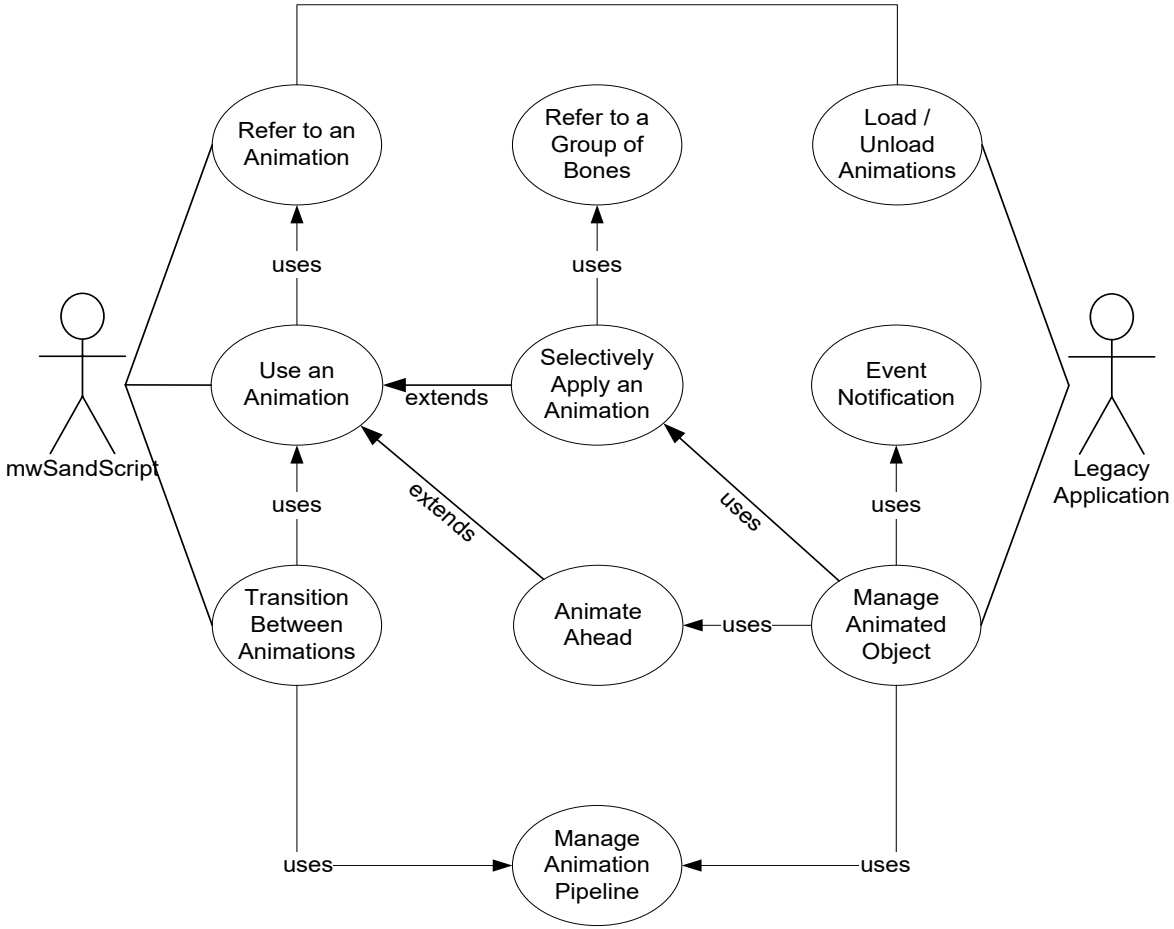


Design Documentation:

UML Use Case Diagrams

- Use cases
  - Technique for capturing the functional requirements of a system
  - Describing the typical interactions between the users of a system and the system itself
    - Providing a narrative of how a system is used
- Rather than describe use cases head-on
  - A ***scenario*** is a sequence of steps describing an interaction between a user and a system.

Design Documentation:
Use Case Diagrams



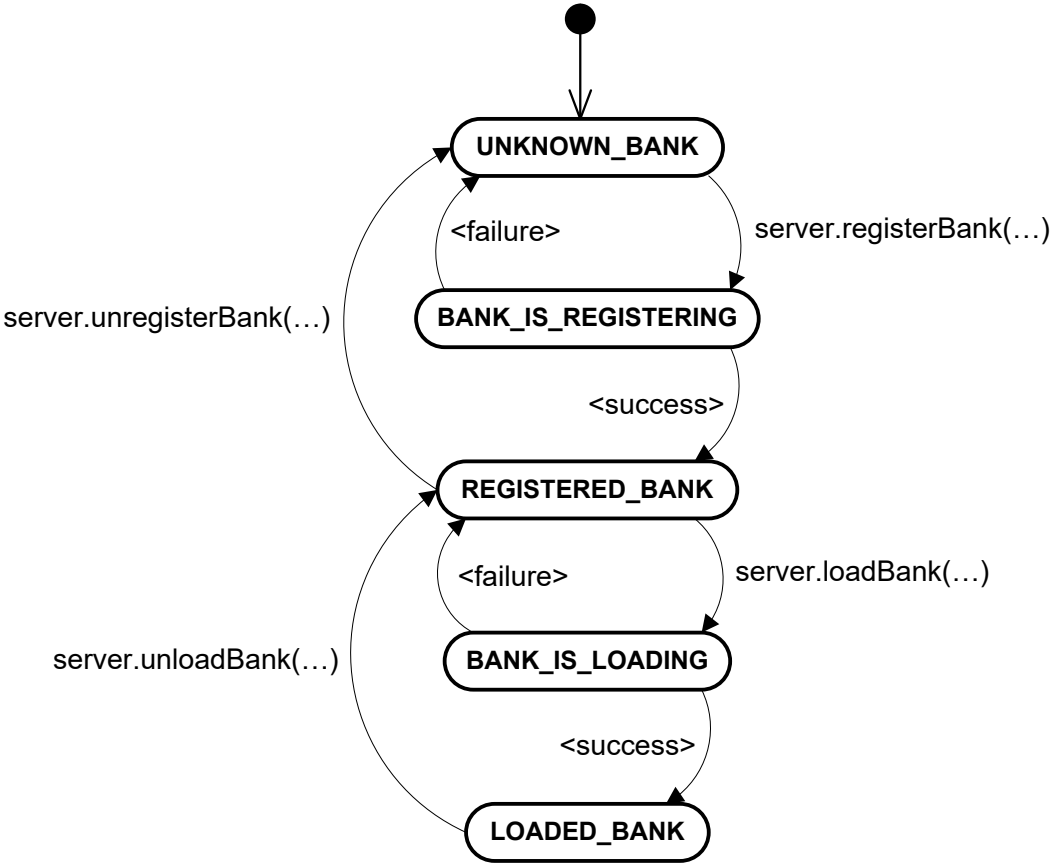
Design Documentation:

UML State Diagrams

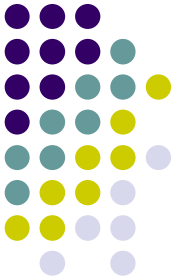
- State machine diagrams
  - Describe the behavior of a system.
- Object-oriented style
  - draw a state machine diagram for a single class
  - show the lifetime behavior of a single object

Design Documentation:

State Machine Diagrams



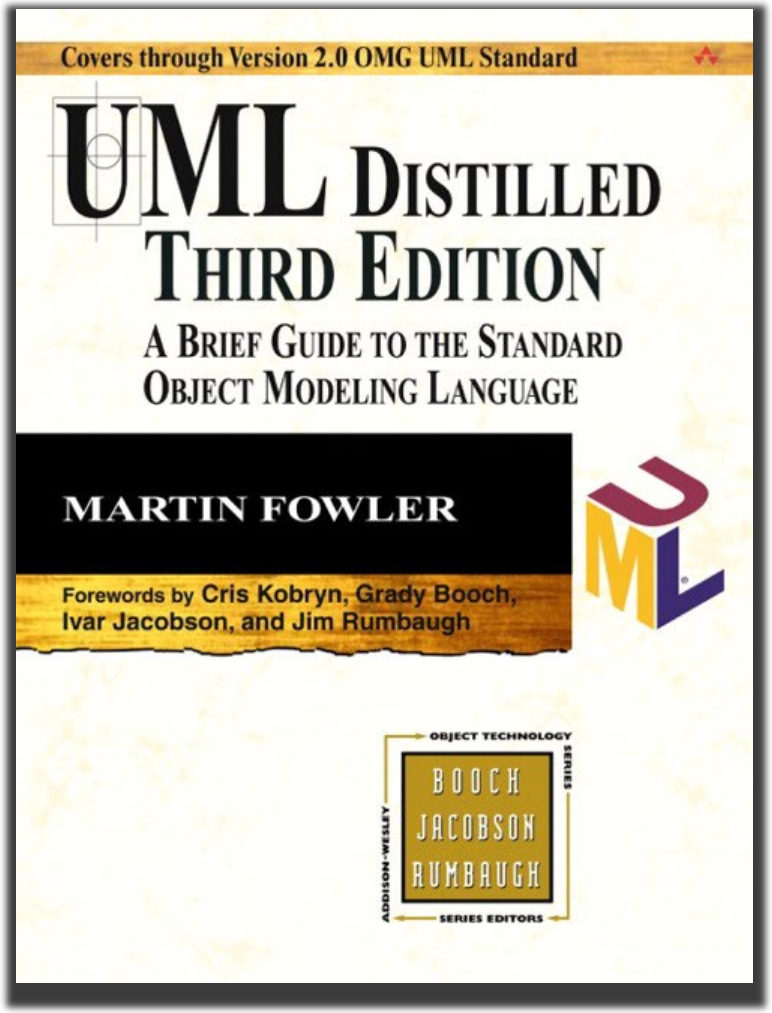
Painless specifications



- Joel on Software
  - Great article on specifications and documentation
    - http://www.joelonsoftware.com/articles/fog0000000036.html
- On this website:
  - Very practical advice and perspective on real world issues.
  - Generally speaking,
    - I agree with this author's philosophy
    - And the book: Pragmatic Programming
      - Great resources, please read

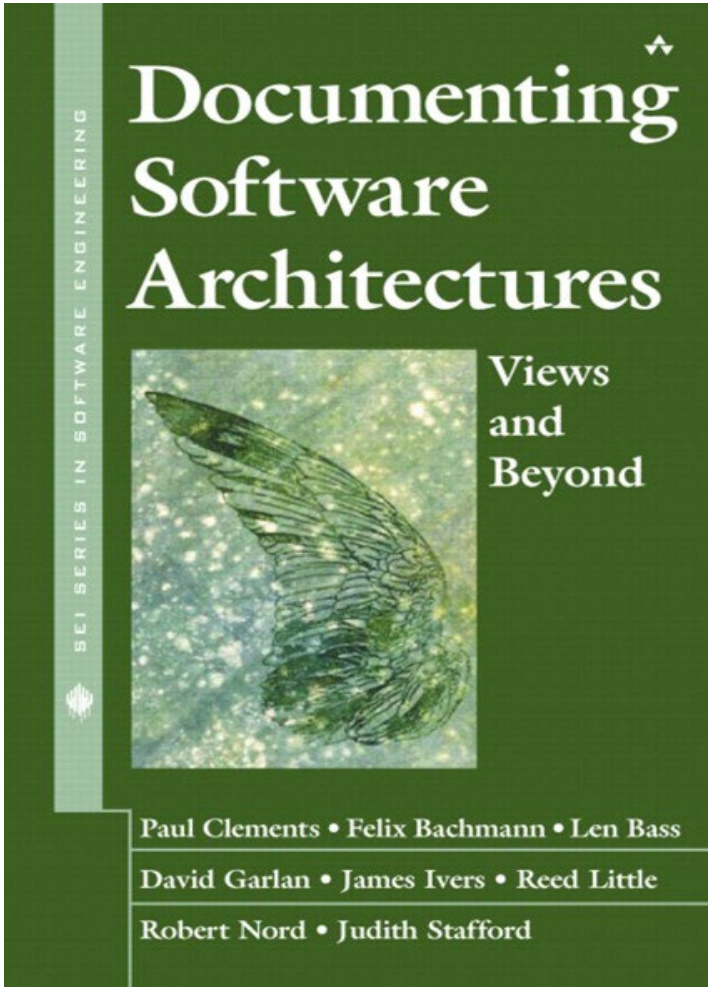
UML Distilled

- Excellent book,
  - Get up and diagramming designs in minutes.
  - Inside cover has a quick cheat sheet or symbols.
- A must if you design or read other people's designs



Documenting Software Architectures

- Very good book
- Describes how do document material that isn't part of the UML spec.
- Gives guidelines on how to define new visual symbols and associate meaning.



Questions

- ????